

```
// forge-check.js - a rough, real first pass at the four-category checker
// Detects: (1) resource exhaustion, (2) environment drift,
//           (3) missing defensive handling, (4) unguarded irreversible actions

const fs = require('fs');

function check(filePath) {
    const code = fs.readFileSync(filePath, 'utf8');
    const findings = [];

    // 1. RESOURCE EXHAUSTION - connection acquired but no matching release/close
    const acquiresConn = /const\s+(\w+)\s*=\s*await\s+\w+\.\getConnection\(\)\.exec(
    if (acquiresConn) {
        const connVar = acquiresConn[1];
        const hasRelease = new RegExp(`${connVar}\\.(release|close|end)\\((`)).test(code);
        const hasFinally = /finally\s*{/\.test(code);
        if (!hasRelease || !hasFinally) {
            findings.push({
                category: 'Resource exhaustion',
                detail: `Connection "${connVar}" is acquired but never released. Under su
                severity: 'Critical',
                fix: `Wrap logic in try/finally and call ${connVar}.release() uncondition
            });
        }
    }

    // 2. ENVIRONMENT DRIFT - hardcoded local/dev-only paths or assumptions
    const hardcodedTmpPath = /["`]\/tmp\/\/.exec(code);
    if (hardcodedTmpPath) {
        findings.push({
            category: 'Environment drift',
            detail: `Writes directly to a hardcoded /tmp path. This works on a local ma
            severity: 'Elevated',
            fix: `Use a configured, environment-aware logging/storage target (e.g. stru
        });
    }

    // 3. MISSING DEFENSIVE HANDLING - external network call with no retry/timeout/
    const externalCallMatch = /await\s+(\w+)\.(\w+)\(\{/.exec(code);
    if (externalCallMatch) {
        const [, obj, method] = externalCallMatch;
        const isExternalLike = /psp|payment|provider|api|client|http/i.test(obj);
        const wrappedInTry = /try\s*{\^[^}]*await\s+\w+\.\w+\(/s.test(code);
        const hasRetryLogic = /retry|backoff|attempt/i.test(code);
```

```

    if (isExternalLike && (!wrappedInTry || !hasRetryLogic)) {
      findings.push({
        category: 'Missing defensive handling',
        detail: `Call to external service "${obj}.${method}()" has no retry/backo
        severity: 'Critical',
        fix: `Wrap the call with retry/backoff and explicit timeout handling. Dis
      });
    }
  }

  // 4. UNGUARDED IRREVERSIBLE ACTION – financial/destructive write with no idemp
  const hasRefundOrDestructive = /refund|delete|drop|wipe|charge/i.test(code);
  const hasIdempotencyGuard = /idempot|already.?(processed|refunded)|dedupe/i.test
  if (hasRefundOrDestructive && !hasIdempotencyGuard) {
    findings.push({
      category: 'Unguarded irreversible action',
      detail: `This performs a financial/destructive action (refund) with no idem
      severity: 'Critical',
      fix: `Check for an existing refund record for this orderId before calling t
    });
  }

  return findings;
}

const findings = check('./before.js');

console.log(`\nForge – reviewed before.js\n${'─'.repeat(60)}`);
console.log(`${findings.length} issue(s) found before this would be safe to merge
findings.forEach((f, i) => {
  console.log(`${i + 1}. [${f.severity}] ${f.category}`);
  console.log(`    ${f.detail}`);
  console.log(`    Fix: ${f.fix}\n`);
});

module.exports = { check };

```